

Time series forecasting

Machine learning methods

Julien JACQUES

Université Lumière Lyon 2

Neural network models

Deep Neural network models

Other Machine Learning models

Neural network models

Neuron

A neuron is a *model*, with p features, which map the p *inputs* $x^1, \dots, x^p$ to an *output* y :

$$y = g \left(\alpha_0 + \sum_{j=1}^p \alpha_j x^j \right)$$

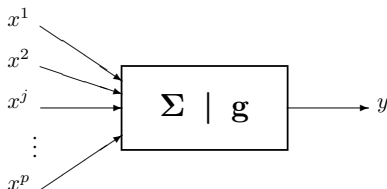


Figure 1: Neuron representation

- ▶ Σ : linear combination of inputs
- ▶ g : activation function

A specific neuron: linear model

One neuron with linear activation function $g(x) = x$ is the usual *linear model*:

$$y = \alpha_0 + \sum_{j=1}^p \alpha_j x^j$$

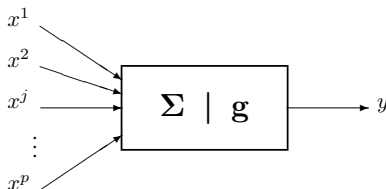


Figure 2: Neuron representation

Neural networks

A neural network is the association of several neurons, in a more or less complex graph, characterized by:

- ▶ its architecture (layer ...)
- ▶ its complexity (number of neurons, presence of loops)
- ▶ activation functions
- ▶ the objective: supervised or unsupervised learning ...

Multilayer perceptron

- ▶ A multilayer perceptron is made up of **layers**
- ▶ Layer: set of neurons without connection between them
- ▶ It has an input layer, an output layer, and **one or more hidden layers**
- ▶ The neurons are all connected at the input to each of the neurons of the previous layer and at the output to each of the neurons of the next layer

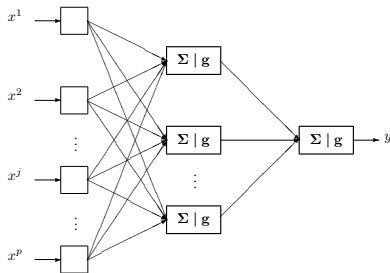


Figure 3: Multilayer perceptron with 1 hidden layer

Neural Network Auto-Regression (NNAR)

For **non seasonal** data:

- ▶ $NNAR_{p,k}$ model:
 - ▶ Inputs: lagged values of the time series $x_{t-1}, \dots, x_{t-p}$
 - ▶ 1 hidden layer with k neurons
 - ▶ sigmoid activation function
 - ▶ Rk: $NNAR_{p,0} = AR_p$

Neural Network Auto-Regression (NNAR)

For **non seasonal** data:

- ▶ $NNAR_{p,k}$ model:
 - ▶ Inputs: lagged values of the time series $x_{t-1}, \dots, x_{t-p}$
 - ▶ 1 hidden layer with k neurons
 - ▶ sigmoid activation function
 - ▶ Rk: $NNAR_{p,0} = AR_p$

For **seasonal** data (of period T), we add lagged values from the same season as last observed values:

- ▶ $NNAR_{(p,P,k)_T}$ model:
 - ▶ Inputs: lagged values of the time series $x_{t-1}, x_{t-2}, \dots, x_{t-p}, x_{t-T}, x_{t-2T}, \dots, x_{t-PT}$
 - ▶ 1 hidden layer with k neurons
 - ▶ sigmoid activation function
 - ▶ Rk: $NNAR_{(p,P,0)_T} = SARIMA_{(p,0,0)(P,0,0)_T}$

nnetar function

Estimation of an $NNAR_{(p,P,k)_T}$ with the forecast package:

```
nnetar(x, p, P, size=k)
```

- ▶ if p not specified, it is chosen automatically by minimizing AIC of a linear AR_p model
- ▶ if P not specified, $P = 1$ is chosen
- ▶ if k not specified, $k = (p + P + 1)/2$ is chosen

Other options:

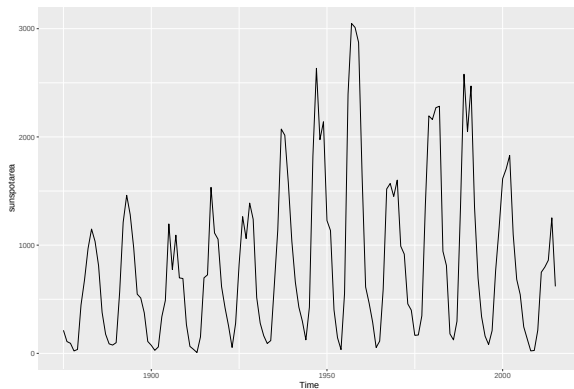
- ▶ `xreg` allows to add external regressors
- ▶ `lambda` allows to use Box-Cox transformation

Neural Network Auto-Regression (NNAR)

- ▶ Advantage over a linear model (AR_p):
 - ▶ more flexible, modeling non-linear relation
- ▶ Dis-advantage over a linear model (AR_p):
 - ▶ none well-defined stochastic model $\rightarrow$ prediction interval not direct (need bootstrap simulations, option PI=TRUE)
 - ▶ not possible to *integrate* differencing

Example: sunspots

```
autoplot(sunspotarea)
```



No seasonal but **cyclic pattern** $\Rightarrow$ can not be modeled by usual linear *SARIMA* models

Example: sunspots

$NNAR_{p,k}$ model estimation, with automatic choice of p and k :

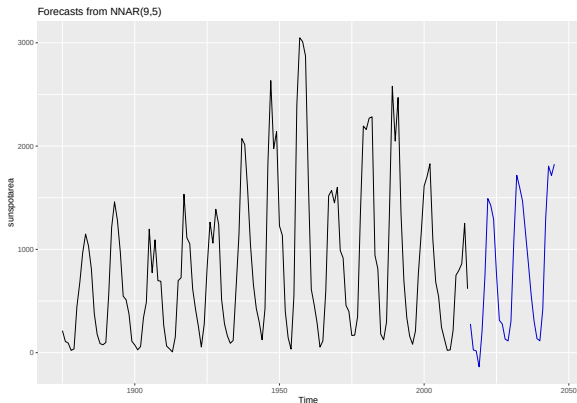
```
fit=nnetar(sunspotarea)
print(fit)
```

```
## Series: sunspotarea
## Model:  NNAR(9,5)
## Call:   nnetar(y = sunspotarea)
##
## Average of 20 networks, each of which is
## a 9-5-1 network with 56 weights
## options were - linear output units
##
## sigma^2 estimated as 9774
```

Example: sunspots

Forecasting for next 30 years:

```
autoplot(forecast(fit,h=30))
```



- asymmetric cyclicity as been modelled well

Deep Neural network models

Recurrent Neural Network

- ▶ More sophisticated neural networks have been developed for time series as RNNs (Recurrent Neural Networks), LSTM, GRU, Transformers.
- ▶ RNNs are designed to process sequences of data. They have a feedback loop that allows them to maintain a memory of previous inputs.

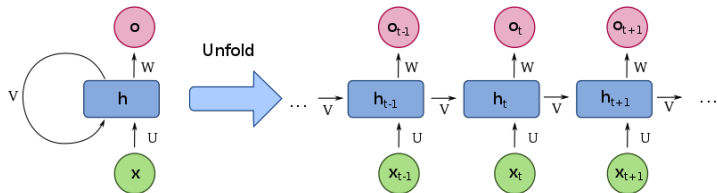


Figure 4: Neuron representation

RNN in R

RNN are implemented in the Keras lib. for Python.

We can use it through the keras R package

```
library(keras3)
```

The idea is to split the whole time series into sub-series.

For instance for weekly data:

- ▶ $x_8 = f(x_7, \dots, x_1)$
- ▶ $x_9 = f(x_8, \dots, x_2)$
- ▶ and so on
- ▶ $x_n = f(x_{n-1}, \dots, x_{n-7})$

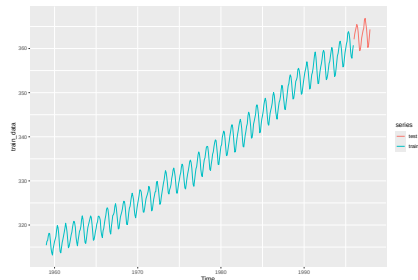
A neural network is learned to estimate f , and then used to forecast x_{n+1}

- ▶ $\hat{x}_{n+1} = \hat{f}(x_n, \dots, x_{n-6})$

co2 forecasting with RNN in R

We test RNN on the co2 time series

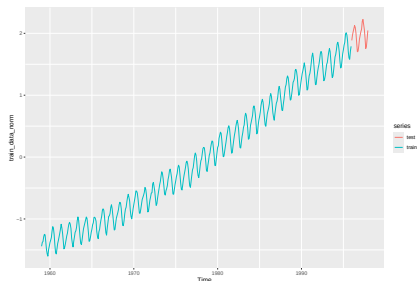
```
train_data <- window(co2, end = c(1995, 12))  
test_data <- window(co2, start = c(1996, 1))  
autoplot(train_data, series="train")+  
  autolayer(test_data, series="test")
```



co2 forecasting with RNN in R

RNN being sensitive to data normalisation, we standardize the time series using the train mean and s.d.:

```
m <- mean(train_data)
s <- sd(train_data)
train_data_norm <- (train_data - m) / s
test_data_norm <- (test_data - m) / s
autoplot(train_data_norm, series="train")+
  autolayer(test_data_norm, series="test")
```



co2 forecasting with RNN in R

RNN training need to build sequences of data:

```
create_sequences <- function(data, window_size) {  
  X <- list()  
  Y <- list()  
  for (i in 1:(length(data) - window_size)) {  
    X[[i]] <- data[i:(i + window_size - 1)]  
    Y[[i]] <- data[i + window_size]  
  }  
  X <- array(unlist(X), dim=c(length(X), window_size, 1))  
  Y <- unlist(Y)  
  return(list(X = X, Y = Y))  
}
```

co2 forecasting with RNN in R

```
train_sequences <- create_sequences(train_data_norm,  
                                   window_size = 12)  
test_sequences  <- create_sequences(test_data_norm,  
                                   window_size = 12)  
  
X_train <- train_sequences$X  
Y_train <- train_sequences$Y  
X_test  <- test_sequences$X  
Y_test  <- test_sequences$Y
```

co2 forecasting with RNN in R

Now, we can build a **RNN** with Keras. We used a SimpleRNN layer.

```
library(keras3)
library(tensorflow)

model <- keras_model_sequential() %>%
  layer_simple_rnn(units = 50, activation = 'tanh',
                  input_shape = c(12, 1)) %>%
  layer_dense(units = 1)
```

- ▶ units = nb of neurons
- ▶ input_shape = c(12, 1) : input data is a vector of length 12 with one feature per time stamp.

The last line adds a dense (or fully connected) layer to the model. The dense layer has a single unit, meaning it produces a single output value.

co2 forecasting with RNN in R

We now can compile the model and print its summary

```
model %>% compile(  
  loss = 'mean_squared_error',  
  optimizer = 'adam'  
)  
summary(model)
```

co2 forecasting with RNN in R

Let's now train the model:

```
history <- model %>% fit(  
  X_train, Y_train,  
  epochs = 50,  
  batch_size = 16,  
  validation_data = list(X_test, Y_test)  
)
```

Epoch 1/50

27/27 - 1s - 46ms/step - loss: 0.8937 - val_loss: 0.2895

Epoch 2/50

27/27 - 0s - 5ms/step - loss: 0.7629 - val_loss: 0.0331

Epoch 3/50

27/27 - 0s - 5ms/step - loss: 0.7299 - val_loss: 0.0573

Epoch 4/50

27/27 - 0s - 5ms/step - loss: 0.6711 - val_loss: 0.0234

Epoch 5/50

27/27 - 0s - 5ms/step - loss: 0.6297 - val_loss: 0.0153

co2 forecasting with RNN in R

Now have to implement the recursive forecasting method

```
predict_recursive <- function(model, initial_sequence,
                               n_ahead = 24, m, s) {
  preds <- numeric(n_ahead)
  input_seq <- initial_sequence
  for (i in 1:n_ahead) {
    input_array <- array(input_seq,
                        dim = c(1,length(input_seq), 1))
    next_pred <- model %>% predict(input_array)
    preds[i] <- next_pred
    input_seq <- c(input_seq[-1],as.numeric(next_pred))
  }
  preds_denorm <- preds * s + m
  return(preds_denorm)
}
```

co2 forecasting with RNN in R

And then we can perform the forecast

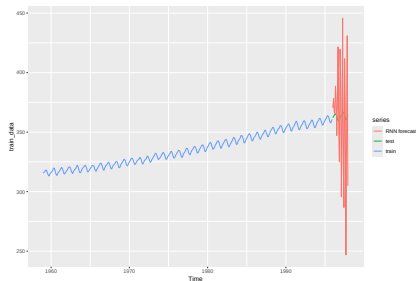
```
initial_seq <- as.numeric(tail(train_data_norm, 12))  
Y_pred_recu <- predict_recursive(model,  
                                initial_seq, n_ahead = 24, m, s)
```

```
## 1/1 - 0s - 128ms/step  
## 1/1 - 0s - 36ms/step  
## 1/1 - 0s - 37ms/step  
## 1/1 - 0s - 33ms/step  
## 1/1 - 0s - 34ms/step  
## 1/1 - 0s - 34ms/step  
## 1/1 - 0s - 33ms/step  
## 1/1 - 0s - 35ms/step  
## 1/1 - 0s - 33ms/step  
## 1/1 - 0s - 34ms/step  
## 1/1 - 0s - 39ms/step  
## 1/1 - 0s - 34ms/step  
## 1/1 - 0s - 33ms/step
```

co2 forecasting with RNN in R

Forecasting results:

```
Y_pred_ts <- ts(Y_pred_recu,  
                start = c(1996, 1), frequency = 12)  
autoplot(train_data, series="train")+  
  autolayer(test_data, series="test")+  
  autolayer(Y_pred_ts, series="RNN forecast")
```



co2 forecasting with RNN in R

We can try to train the model for estimating directly the 24 values.

```
create_multistep_sequences <- function(data, input_size,
                                       output_size) {
  X <- list()
  Y <- list()
  for (i in 1:(length(data)-input_size-output_size+1)){
    X[[i]] <- data[i:(i + input_size - 1)]
    Y[[i]] <- data[(i + input_size):(i + input_size
                                     + output_size-1)]
  }
  X <- array(unlist(X), dim = c(length(X),input_size,1))
  Y <- array(unlist(Y), dim = c(length(Y),output_size))
  return(list(X = X, Y = Y))
}
```

co2 forecasting with RNN in R

We define the model for a vector of length 24 in inputs

```
model <- keras_model_sequential() %>%  
  layer_simple_rnn(units = 50, activation = 'tanh',  
                   input_shape = c(48, 1)) %>%  
  layer_dense(units = 24)
```

co2 forecasting with RNN in R

We extract the sequences

```
sequences <- create_multistep_sequences(train_data_norm,  
                                         input_size = 48,  
                                         output_size = 24)  
  
X_train <- sequences$X  
Y_train <- sequences$Y
```

co2 forecasting with RNN in R

We can train the model

```
model %>% compile(  
  loss = 'mean_squared_error',  
  optimizer = 'adam'  
)  
history <- model %>% fit(  
  X_train, Y_train,  
  epochs = 50,  
  batch_size = 16  
)
```

```
## Epoch 1/50
```

```
## 24/24 - 1s - 50ms/step - loss: 0.4563
```

```
## Epoch 2/50
```

```
## 24/24 - 0s - 7ms/step - loss: 0.0657
```

```
## Epoch 3/50
```

```
## 24/24 - 0s - 7ms/step - loss: 0.0305
```

```
## Epoch 4/50
```

co2 forecasting with RNN in R

The we estimate the forecast

```
initial_seq <- tail(train_data_norm, 48)
input_array <- array(initial_seq, dim = c(1, 12, 1))
Y_pred_norm <- model %>% predict(input_array)
```

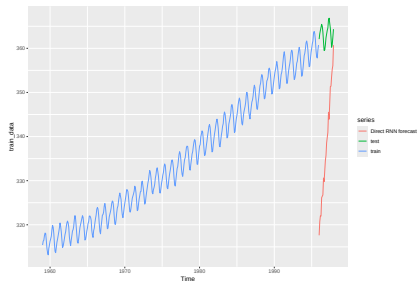
```
## 1/1 - 0s - 131ms/step
```

```
Y_pred <- as.vector(Y_pred_norm * s + m)
```


co2 forecasting with RNN in R

Forecasting results representation

```
Y_pred_ts <- ts(Y_pred[], start = c(1996, 1),  
               frequency = 12)  
autoplot(train_data, series="train") +  
  autolayer(test_data, series="test") +  
  autolayer(Y_pred_ts, series="Direct RNN forecast")
```



Exercise: San Francisco precipitation

San Fransisco precipitation from 1932 to 1966 are available here:
<http://eric.univ-lyon2.fr/jjacques/Download/DataSet/sanfran.dat>

- ▶ Try to improve your forecasts obtained with exponential smoothing and SARIMA models with neural network models

Exercices:

Try to improve all your previous forecast with NN ! ... good luck ;-)

Other Machine Learning models

Data prepration

- ▶ any machine learning model can be tune once we correctly split the time series $x_1, \dots, x_n$ into input|output data set:

<i>inputs</i>			<i>output</i>
x_1	$\dots$	x_T	x_{T+1}
x_{n-T-1}	$\dots$	x_{n-2}	x_{n-1}
x_{n-T}	$\dots$	x_{n-1}	x_n

- ▶ then, we just have to learn any function f :

$$x_n = f(x_{n-T}, \dots, x_{n-1})$$

- ▶ f can be Random Forest, Gradient boosting, SVM, LM...

Data preparation for San Francisco data set

```
data=scan(file="data/sanfran.csv",skip=1)
sanfran<-ts(data,start=c(1932,1),end=c(1966,12),freq=12)
library(forecast)
sanfran_train=window(sanfran,start=c(1932,1),end=c(1963,12))
sanfran_test=window(sanfran,start=c(1964,1),end=c(1966,12))
data=as.vector(sanfran_train)[1:13]
for (i in 1:(length(as.vector(sanfran_train))-13)){
data=rbind(data,as.vector(sanfran_train)[(i+1):(i+13)])
}
print(head(sanfran))
```

```
##           Jan   Feb   Mar   Apr   May   Jun
## 1932 16.26 29.46 18.03 24.13 22.35 22.10
```

```
print(data[1:2,1:6])
```

```
##           [,1] [,2] [,3] [,4] [,5] [,6]
## data 16.26 29.46 18.03 24.13 22.35 22.10
##           29.46 18.03 24.13 22.35 22.10 12.95
```

Random Forest

We fit the model

```
library(randomForest)
fitRF=randomForest(x=data[, -13], y=data[, 13])
```

And then sequentially forecast the next 36 values

```
pred=rep(NA, 36)
newdata=tail(sanfran_train, 12)
for (t in 1:36){
  pred[t]=predict(fitRF, newdata=newdata)
  newdata=c(newdata[-1], pred[t])
}
prevRF=ts(pred, start=c(1964, 1), end=c(1966, 12), frequency = 12)
```

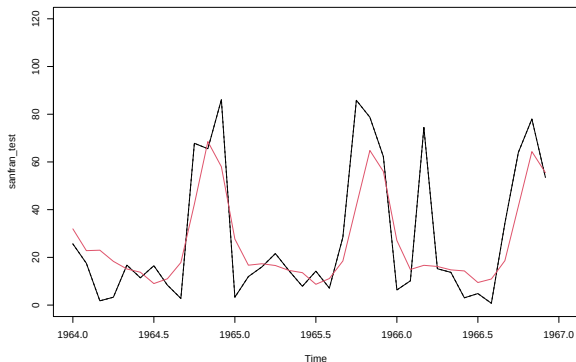
Random Forest

Forecasting results

```
print(sqrt(mean((prevRF-sanfran_test)^2)))
```

```
## [1] 17.06046
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevRF,col=2)
```



Gradient Boosting

Random forest is an aggregation of independent decision/regression trees built on bootstrap samples (*bagging*).

Boosting method consists of aggregating non independent models (typ. trees):

- ▶ a first model is trained on the whole data set
- ▶ a second model is then trained on the same data set, but by given a larger weight to the bad predicted observations
- ▶ and so on. . .

Several boosting model exists: AdaBoost, LPBoost, XGBoost, GradientBoost, BrownBoost. . .

Gradient Boosting

Here we use the probably most known boosting method: XGBoost

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.1
```

Many parameters have to be tune, among which:

- ▶ `max_depth`: maximum depth of a tree
- ▶ $0 < \text{eta} < 1$: the learning rate (scale the contribution of each tree)
- ▶ `nrounds`: maximum number of boosting iterations.
- ▶ `objective`: learning loss (*reg:squarederror* or *reg:squaredlogerror* for regression).

```
model <- xgboost(data = data[,1:12], label = data[,13],  
                 max_depth = 10, eta = .5, nrounds = 100,  
                 nthread = 2, objective = "reg:squarederror")
```

```
## [1] train-rmse:21.074956
```

```
## [2] train-rmse:13.236779
```

```
## [3] train-rmse:8.915509
```

```
## [4] train-rmse:6.398479
```

Gradient Boosting

Now we will do forecasting sequentially

```
pred=rep(NULL,36)
newdata=tail(sanfran_train,12)
for (t in 1:36){
  pred[t]=predict(model,matrix(newdata,1,12))
  newdata=c(newdata[-1],pred[t])
}
prevXGBBOOST=ts(pred,start=c(1964,1),end=c(1966,12),frequency=12)
```

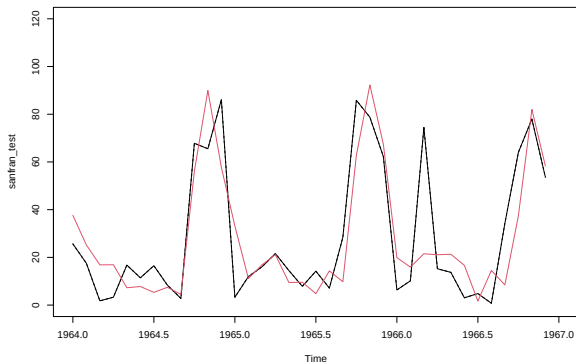
Gradient Boosting

Forecasting results

```
print(sqrt(mean((prevXGB00ST-sanfran_test)^2)))
```

```
## [1] 16.19329
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevXGB00ST,col=2)
```



SVM

Model fitting and sequential forecasting

```
library(e1071)
fitSVM=svm(x=data[,-13], y=data[,13])
pred=rep(NULL,36)
newdata=tail(sanfran_train,12)
for (t in 1:36){
  pred[t]=predict(fitSVM,newdata=matrix(newdata,1,12))
  newdata=c(newdata[-1],pred[t])
}
prevSVM=ts(pred,start=c(1964,1),end=c(1966,12),frequency =
```

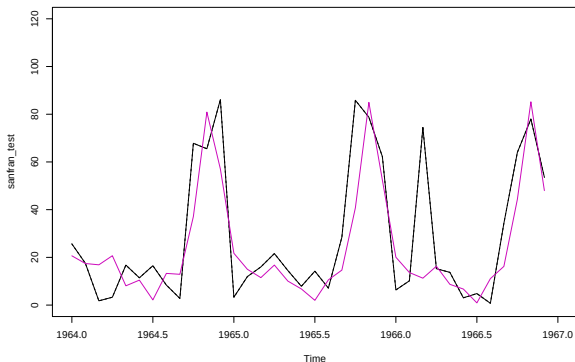
SVM

Forecasting results

```
print(sqrt(mean((prevSVM-sanfran_test)^2)))
```

```
## [1] 17.53055
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevSVM,col=6)
```



LM

Model fitting and sequential forecasting

```
data=data.frame(data)
colnames(data)=c('x1','x2','x3','x4','x5','x6','x7','x8','x9')
fitLM=lm(y~.,data=data)
pred=rep(NULL,36)
newdata=data.frame(matrix(tail(sanfran_train,12),1,12))
colnames(newdata)=c('x1','x2','x3','x4','x5','x6','x7','x8')
for (t in 1:36){
  pred[t]=predict(fitLM,newdata=newdata)
  newdata=data.frame(c(newdata[-1],pred[t]))
  colnames(newdata)=c('x1','x2','x3','x4','x5','x6','x7','x8')
}
prevLM=ts(pred,start=c(1964,1),end=c(1966,12),frequency = 12)
```

Rk: be careful, lm assume i.i.d. observation, what is wrong here

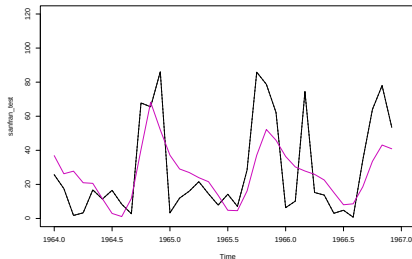
LM

Forecasting results

```
print(sqrt(mean((prevLM-sanfran_test)^2)))
```

```
## [1] 20.49199
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevLM,col=6)
```



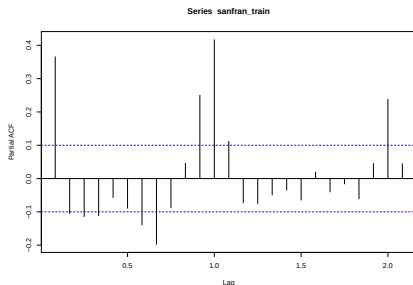
Which inputs ?

In the previous application, we used the 12 previous observations.

$$x_n = f(x_{n-1}, \dots, x_{n-12})$$

Is-it a good idea?

```
pacf(sanfran_train)
```



Looking at the PACF can advice us to use:

$$x_n = f(x_{n-1}, x_{n-8}, x_{n-11}, x_{n-12}, x_{n-24})$$

New data preparation

```
data=as.vector(sanfran_train)[c(1,13,14,17,24,25)]
for (i in 1:(length(as.vector(sanfran_train))-25)){
data=rbind(data,as.vector(sanfran_train)[i+c(1,13,14,17,24,25)])
}
print(str(data))
```

```
##  num [1:360, 1:6] 16.3 29.5 18 24.1 22.4 ...
##  - attr(*, "dimnames")=List of 2
##    ..$ : chr [1:360] "data" "" "" "" ...
##    ..$ : NULL
##  NULL
```

RF with new inputs

```
library(randomForest)
fitRF=randomForest(x=data[, -6], y=data[, 6])
```

And then sequentially forecast the next 36 values

```
pred=rep(NULL, 36)
newdata=sanfran_train[length(sanfran_train)-c(1,8,11,12,24)]
newsf=sanfran_train
for (t in 1:36){
  pred[t]=predict(fitRF, newdata=newdata)
  newsf=c(newsf, pred[t])
  newdata=newsf[length(newsf)-c(1,8,11,12,24)]
}
prevRF=ts(pred, start=c(1964,1), end=c(1966,12), frequency = 12)
```

RF with new inputs

Forecasting results

```
print(sqrt(mean((prevRF-sanfran_test)^2)))
```

```
## [1] 28.4357
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevRF,col=6)
```



To go further

- ▶ forecasting is done sequentially : $\hat{x}_{n+2}$ uses $\hat{x}_{n+1}$.
- ▶ improvement would be to:
 - ▶ forecast directly $\hat{x}_{n+2}$
 - ▶ simultaneously forecast $(\hat{x}_{n+1}, \hat{x}_{n+2})$
- ▶ for this machine learning model with multivariate output have to be used
 - ▶ but not so much models are developed for this task

Multivariate LM

We implement a LM model with multivariate output.

We consider 1 year (12 obs) as output, and 2 years as inputs (24 obs).

```
data=as.vector(sanfran_train)[1:36]
for (i in 1:(length(as.vector(sanfran_train))-36)){
  data=rbind(data,as.vector(sanfran_train)[(i+1):(i+36)])
}
data=data.frame(data)
for (i in 1:36) colnames(data)[i]=paste('x',i,sep='')
fitLM=lm(cbind(x36,x35,x34,x33,x32,x31,x30,x29,x28,x27,x26,x25)~.,data=data)
```

Multivariate LM

Sequential forecasting by year

```
pred=rep(NULL,36)
newdata=data.frame(matrix(tail(sanfran_train,24),1,24))
for (i in 1:24) colnames(newdata)[i]=paste('x',i,sep='')
pred[1:12]=predict(fitLM,newdata=newdata)
for (j in 1:2){
  newdata=data.frame(c(newdata[-(1:12)],pred[(12*(j-1)+1):(12*j)]))
  for (i in 1:24) colnames(newdata)[i]=paste('x',i,sep='')
  pred[(12*(j)+1):(12*(j+1))]=predict(fitLM,newdata=newdata)
}
prevMLM=ts(pred,start=c(1964,1),end=c(1966,12),frequency = 12)
```

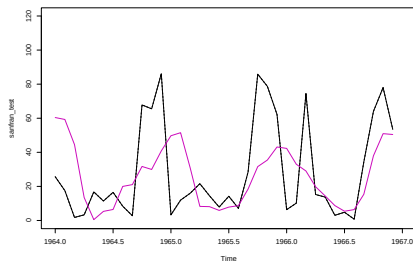
Multivariate LM

Forecasting results

```
print(sqrt(mean((prevMLM-sanfran_test)^2)))
```

```
## [1] 26.66568
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevMLM,col=6)
```



Multivariate LM

We can change by sequentially forecasting months by months and not year by year

```
pred=rep(NULL,36)
newdata=data.frame(matrix(tail(sanfran_train,24),1,24))
for (i in 1:24) colnames(newdata)[i]=paste('x',i,sep='')
pred[1:12]=predict(fitLM,newdata=newdata)
for (j in 1:35){
  newdata=data.frame(c(newdata[-1],pred[j]))
  for (i in 1:24) colnames(newdata)[i]=paste('x',i,sep='')
  pred[j+1]=predict(fitLM,newdata=newdata)[1]
}
prevMLM=ts(pred,start=c(1964,1),end=c(1966,12),frequency = 12)
```

Multivariate LM

Forecasting results

```
print(sqrt(mean((prevMLM-sanfran_test)^2)))
```

```
## [1] 33.72657
```

```
plot(sanfran_test,xlim=c(1964,1967),ylim=c(0,120))  
lines(sanfran_test,lty=2)  
lines(prevMLM,col=6)
```

